GK7.1 – REST, OpenAPI und Datenformate

Start und Vorführung

```
./gradlew test
./gradlew bootRun
```

Webclient: `http://localhost:8080/`. Im Client Station wählen, Zeitraum/Warnstufe filtern, JSON/XML umschalten und automatische Aktualisierung beobachten. API-Dokumentation: `/swagger-ui/index.html`, maschinenlesbar: `/v3/api-docs`.

```
curl -H 'Accept: application/json' http://localhost:8080/api/v1/stations
curl -H 'Accept: application/xml' http://localhost:8080/api/v1/stations/ST-001/measurements/latest
curl -G 'http://localhost:8080/api/v1/stations/ST-001/measurements' \
  --data-urlencode 'from=2026-11-03T00:00:00Z' \
  --data-urlencode 'to=2026-11-03T23:59:59Z' \
  --data-urlencode 'limit=100'
curl 'http://localhost:8080/api/v1/stations?river=Donau&isActive=true'
curl 'http://localhost:8080/api/v1/alerts?warningLevel=CRITICAL'
curl -i 'http://localhost:8080/api/v1/stations/ST-999'
curl -i -H 'Accept: text/csv' 'http://localhost:8080/api/v1/stations'
```

Die Zeitraumbeispiele beziehen sich auf diesen Tag. Die Anwendung hält Daten nur im Arbeitsspeicher; bei einem späteren Start einen passenden Zeitraum wählen.

1. Hauptmerkmale von JSON

JSON ist ein textbasiertes, sprachunabhängiges Datenaustauschformat. Es unterstützt Objekte mit Schlüssel-Wert-Paaren, geordnete Arrays, Zeichenketten, Zahlen, `true`, `false` und `null`. Schlüssel und Zeichenketten stehen in doppelten Anführungszeichen. Kommentare sind nicht Teil der JSON-Grammatik. Datumswerte haben keinen eigenen JSON-Datentyp; hier werden sie als ISO-8601-Zeichenketten übertragen. Vorteil: kompakt, in JavaScript direkt verarbeitbar. Ein XML-ähnlicher Attribut-/Namensraummechanismus ist nicht eingebaut.

Quelle: RFC 8259, Abschnitte 1–7. Quellcode: `model/Measurement.java` und die JSON-Ausgabe der Controller.

2. Hauptmerkmale von XML

XML beschreibt hierarchische Dokumente mit Elementen, Attributen und Text. Ein wohlgeformtes Dokument hat genau ein Wurzelelement; Tags müssen korrekt verschachtelt und geschlossen sein. XML ist erweiterbar und eignet sich auch für dokumentorientierte Inhalte. Es unterstützt unter anderem Kommentare, Zeichenreferenzen und Validierung mit einer DTD; XML Schema ist eine eigene zusätzliche Spezifikation. Namensräume sind ebenfalls gesondert standardisiert. Gegenüber JSON ist XML meist ausführlicher. Externe Entitäten sollte ein Parser bei nicht vertrauenswürdigen Dokumenten nicht auflösen.

Quellen: XML 1.0, insbesondere Abschnitte 2 und 3, Namespaces in XML. Quellcode: XML-Annotationen im Paket `model` und XML-Verarbeitung im Webclient.

3. Auswahl zwischen JSON und XML

Der Client sendet `Accept: application/json` oder `Accept: application/xml`. Spring MVC wählt einen passenden HTTP-Message-Converter: Jackson für JSON, Jackson XML für XML. Beide serialisieren dieselben fachlichen Daten; der Controller muss nicht zwei Versionen der Geschäftslogik enthalten. Ohne `Accept` ist JSON der Standard. Kann kein angebotenes Format den Header erfüllen, folgt `406 Not Acceptable`. `Accept` beschreibt das gewünschte Antwortformat; `Content-Type` beschreibt dagegen das Format des tatsächlich übertragenen Bodys.

Quellen: RFC 9110, Abschnitt 12.5.1, Spring MVC Message Converters. Quellcode: Controller, Webkonfiguration und `build.gradle`.

4. REST und Verbreitung

REST ist ein Architekturstil, kein bestimmtes Datenformat und kein eigenes Netzwerkprotokoll. Wesentliche Bedingungen sind Client-Server-Trennung, zustandslose Anfragen, Cachefähigkeit, einheitliche Schnittstelle, Schichten und optional Code-on-Demand. Ressourcen werden identifiziert und über Repräsentationen

bearbeitet. HTTP liefert dafür bekannte Methoden, Statuscodes, Header und Caching. Dadurch können sehr unterschiedliche Clients und Server zusammenarbeiten. JSON über HTTP allein macht eine API noch nicht vollständig REST-konform; dieses Projekt implementiert eine ressourcenorientierte HTTP-API, aber kein vollständiges Hypermedia-Navigationsmodell.

Quellen: Fielding, Dissertation, Kapitel 5, RFC 9110. Quellcode: `controller/StationController.java`.

5. GET, POST, PUT und PATCH

Methode	Bedeutung	Sicherheit / Idempotenz
GET	Eine Repräsentation abrufen; keine beabsichtigte Zustandsänderung.	Sicher und idempotent.
POST	Den Body von der Zielressource verarbeiten lassen, häufig eine untergeordnete Ressource erzeugen.	Nicht sicher; nicht allgemein idempotent.
PUT	Zustand der bekannten Zielressource durch die Repräsentation ersetzen oder dort neu anlegen.	Nicht sicher, aber idempotent.
PATCH	Eine Menge partieller Änderungen anwenden.	Nicht sicher; nicht allgemein idempotent, kann entsprechend gestaltet werden.

Idempotent bedeutet: Wiederholung derselben Anfrage hat dieselbe beabsichtigte Wirkung wie einmaliges Ausführen; die Antworten müssen nicht identisch sein. Bei Erstellung sind `201 Created` und eine `Location`-Adresse passend. Diese Grundkompetenzlösung stellt nur lesende GET-Endpunkte bereit, keine Stationsverwaltung.

Quellen: RFC 9110, Abschnitte 9.2 und 9.3, RFC 5789, Abschnitt 2.

6. Neue Request-Methode oder neuer Pfad

1. Im Controller eine passende Mapping-Methode ergänzen, zum Beispiel `@GetMapping`, `@PostMapping` oder `@PatchMapping`. 2. Parameter bzw. Request-DTO und Validierung festlegen. Statuscode und Antwortmodell auswählen. 3. Fachliche Verarbeitung im Service ergänzen; bei neuem Speicherzugriff das Repository erweitern, bei neuen Daten das Model. 4. Passende Fehlerfälle in der zentralen Fehlerbehandlung abdecken. 5. OpenAPI-Beschreibung und automatisierte Tests ergänzen; bei Bedarf den Webclient anpassen.

Für einen reinen neuen lesenden Pfad ist keine Datenmodelländerung nötig, wenn vorhandene Modelle genügen. Die Schichten werden also nach Bedarf und nicht pauschal alle geändert.

Quellen: Spring MVC Request Mapping, OpenAPI 3.0.3 – Operation Object. Quellcode: Pakete `controller`, `service`, `repository`, `model`, `exception` sowie `src/test`.

7. OpenAPI und Swagger

Die Begriffe hängen zusammen, bezeichnen aber nicht dasselbe: OpenAPI ist die herstellerunabhängige Spezifikation zur Beschreibung von HTTP-APIs. Ein OpenAPI-Dokument beschreibt etwa Pfade, Operationen, Parameter, Schemas, Antworten und Sicherheitsverfahren. Swagger ist eine Werkzeugfamilie, beispielsweise Swagger UI zur interaktiven Darstellung, Swagger Editor und Swagger Codegen. Historisch hieß die Spezifikation Swagger Specification; daraus entstand OpenAPI. Ein Vergleich ist sinnvoll als Standard gegenüber Werkzeugen, nicht als zwei konkurrierende Datenformate. Hier erzeugt springdoc das OpenAPI-Dokument; Swagger UI stellt es im Browser dar.

Quellen: OpenAPI 3.0.3, Einleitung, Swagger – OpenAPI Specification, springdoc.

8. Fehler im GET-Beispiel

```
@GetMapping("/stations/add")
public String createStation(@RequestParam String name, @RequestParam double warning) {
    stationService.save(new Station(name, warning));
    return "Station successfully created!";
}
```

Die Methode verändert mit save den Zustand, ist aber als GET deklariert. GET muss sicher sein: Crawler, Link-Vorschauen und Caches dürfen durch Abrufen keine neue Station erzeugen. Außerdem beschreibt `/stations/add` eine Aktion statt einer Ressource. Passend wäre `POST /api/v1/stations` mit validiertem Request-Body, einer vollständigen Station bzw. einem passenden DTO und bei Erfolg `201 Created` samt `Location`. Die angezeigte Erfolgsmeldung ersetzt weder einen passenden Statuscode noch sinnvolle Ressourcendaten. Zusätzlich würde der dargestellte Konstruktor in dieser Lösung nicht kompilieren: Für eine Station werden weitere Angaben benötigt. Der grundsätzliche HTTP-Fehler besteht jedoch unabhängig davon.

Quellen: RFC 9110, 9.2.1 – Safe Methods, 9.3.3 – POST, 15.3.2 – 201 Created.

Für das Abgabegespräch

- Station = Stammdaten und Schwellwerte; Measurement = zeitbezogener Messwert mit `Instant`.

- Wasserstand ab Warnschwelle: `WARNING`; ab kritischer Schwelle: `CRITICAL`; ohne gültige aktuelle Messung: `UNKNOWN`.

- Die Simulation nutzt den vorherigen Messwert; ein HTTP-Abruf erzeugt keine neue Messung.

- Historische Daten werden vor dem Limit gefiltert. Grenzen und ISO-Zeitpunkte werden serverseitig validiert.

- XML und JSON sind zwei Darstellungen derselben Daten, keine zwei getrennten APIs.

- Das System ist ein Simulator, kein verlässliches reales Hochwasserwarnsystem.

- KI-Unterstützung und tatsächlich verwendete Arbeitsaufträge sind in `AI_PROMPTS.md` dokumentiert.